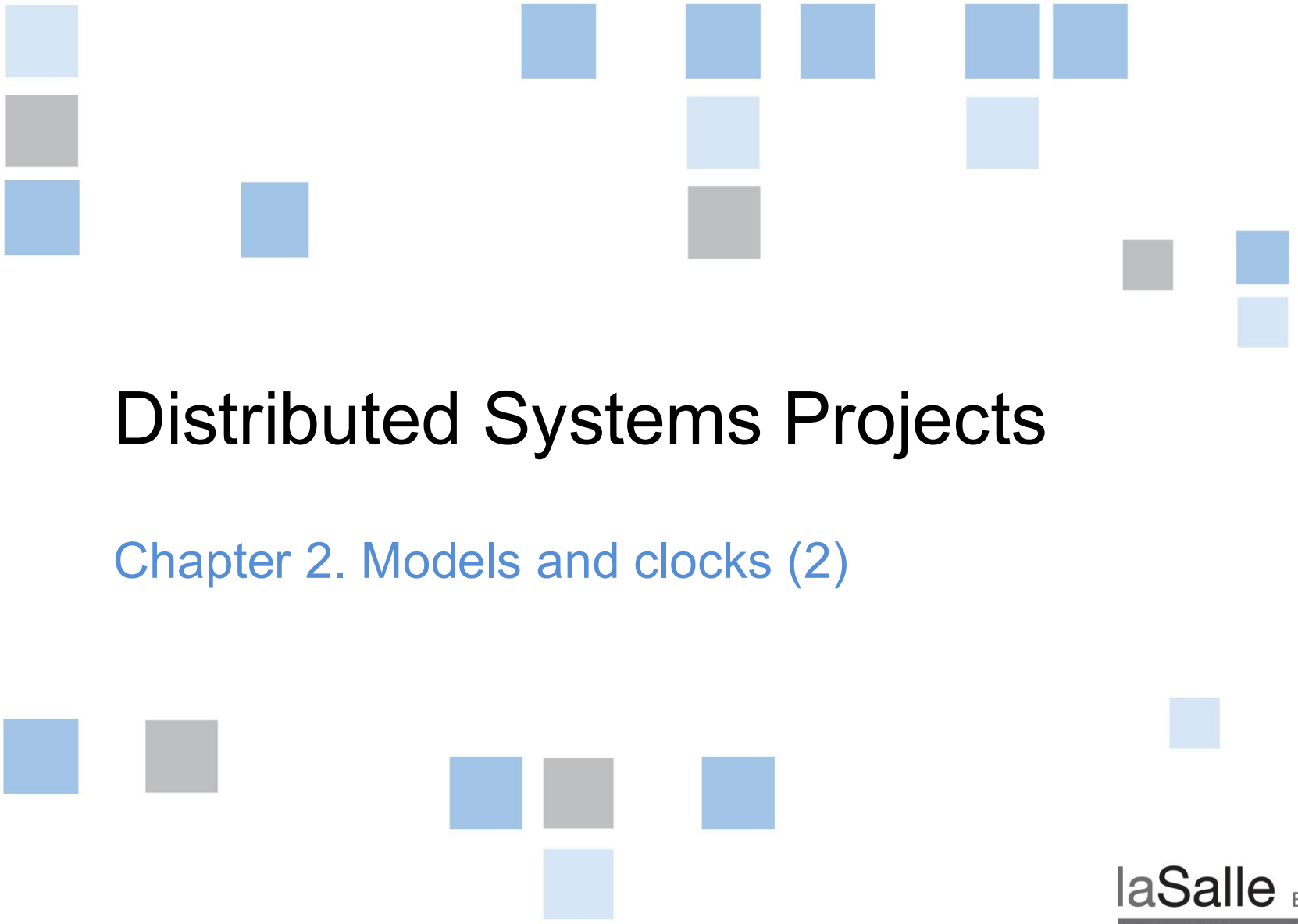


Distributed Systems Projects

Chapter 2. Models and clocks (2)

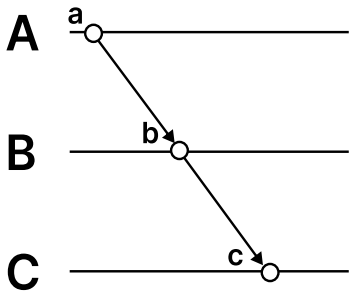
laSalle ENG
Universitat Ramon Llull

Today's agenda:

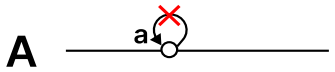
- Network Time Protocol (NTP)
- Distributed system events
- Happens-before (Causality) model
- Logical clocks
  - Lamport clock
  - Vector clock
  - Direct dependency clock

Recap: Happens-before model

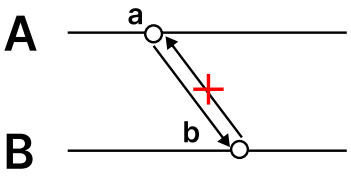
T



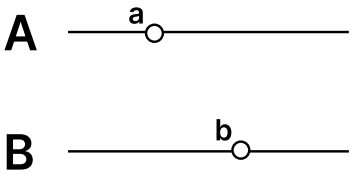
I



A



C



Recap: Happens-before model

The relation is **transitive**

$$\forall a, b, c,\ if\ a \rightarrow b,\ and\ b \rightarrow c, then\ a \rightarrow c$$

The relation is **irreflexive –** an event cannot precede itself

$$\forall a,\ a \not\rightarrow a$$

The relation is **asymmetric**

$$\forall a, b,\ if\ a \rightarrow b\ then\ b \not\rightarrow a$$

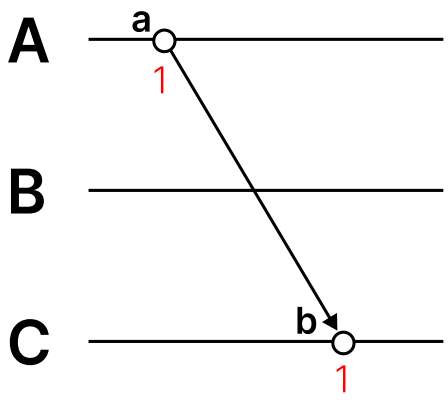
Two distinct events a and b are **concurrent** (||) if:

$$a \not\rightarrow b\ and\ b \not\rightarrow a$$

Recap: Logical clocks

A **logical clock** must satisfy the following constraint:

$$\forall a, b \in E : a \rightarrow b \implies t(a) < t(b)$$



Logical clocks: Vector clock

Lamport logical clock is represented with a single integer.

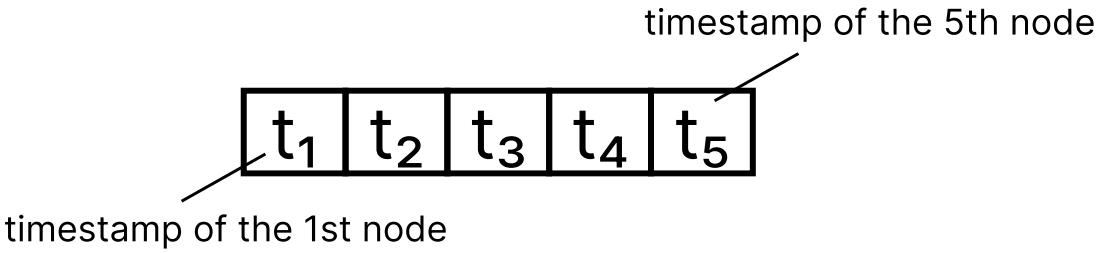
While **vector clocks** are a list of integers, one for each node in the cluster.

- Assume *n* nodes in a cluster

  $Cluster = \{n_1, n_2, n_3, \ldots, n_n\}$

- Then, a vector clock

  $vc = \langle t_1, t_2, t_3, \ldots, t_n \rangle$



In practice, this vector is normally implemented as a **map**.

Node ID	Time
n_1	t_1
n_2	t_2
n_3	t_3
…	…
n_n	t_n

Logical clocks: Vector clock comparison

Vx equal Vy (Same event)

Two VC are considered equal if every corresponding element in the vectors is the same across all nodes involved. In other words, for two vector clocks Vx and Vy, the following condition must be true for all indices i:

$$Vx1[i] = Vy2[i]$$

```
If:
    Vx = [2,3,1]
    Vy = [2,3,1]

// Vx and Vy are the same event
```

Logical clocks: Vector clock comparison

Vx happens before Vy (Vx → Vy)

Each component of *Vx*'s clock is less **than or equal** (<=) to the corresponding element of *Vy*'s clock and at least one component is less than *Vy*'s.

```
If:
    Vx = [2, 3, 1]
    Vy = [3, 5, 2]

// Since for any i, Vx[i] <= Vy[i], Vx is less than Vy then Vx happened before Vy

Similarly if
    Vx = [1, 5, 2]
    Vy = [3, 5, 2]

// At least one component of Vx < Vy, then Vx happened before Vy.
```

Logical clocks: Vector clock comparison

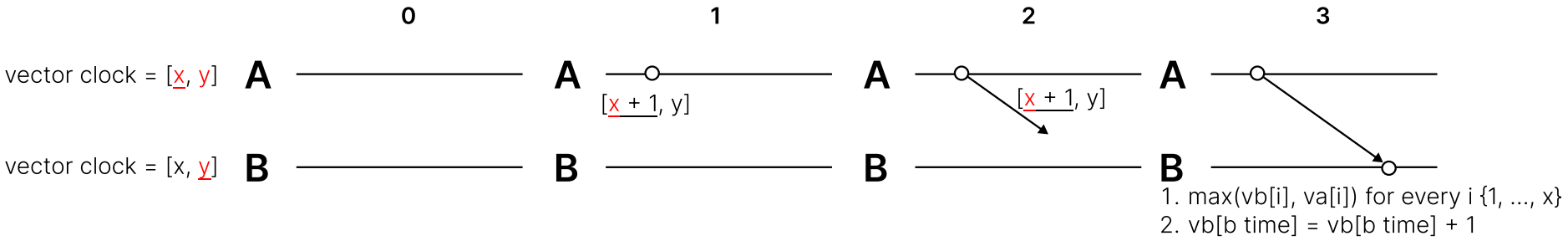
***Vx* and *Vy are concurrent* (Vx || Vy)**:

- at least one index **i** where **Vx[i] > Vy[i]**
- at least one index **j** where **Vx[j] < Vy[j]**

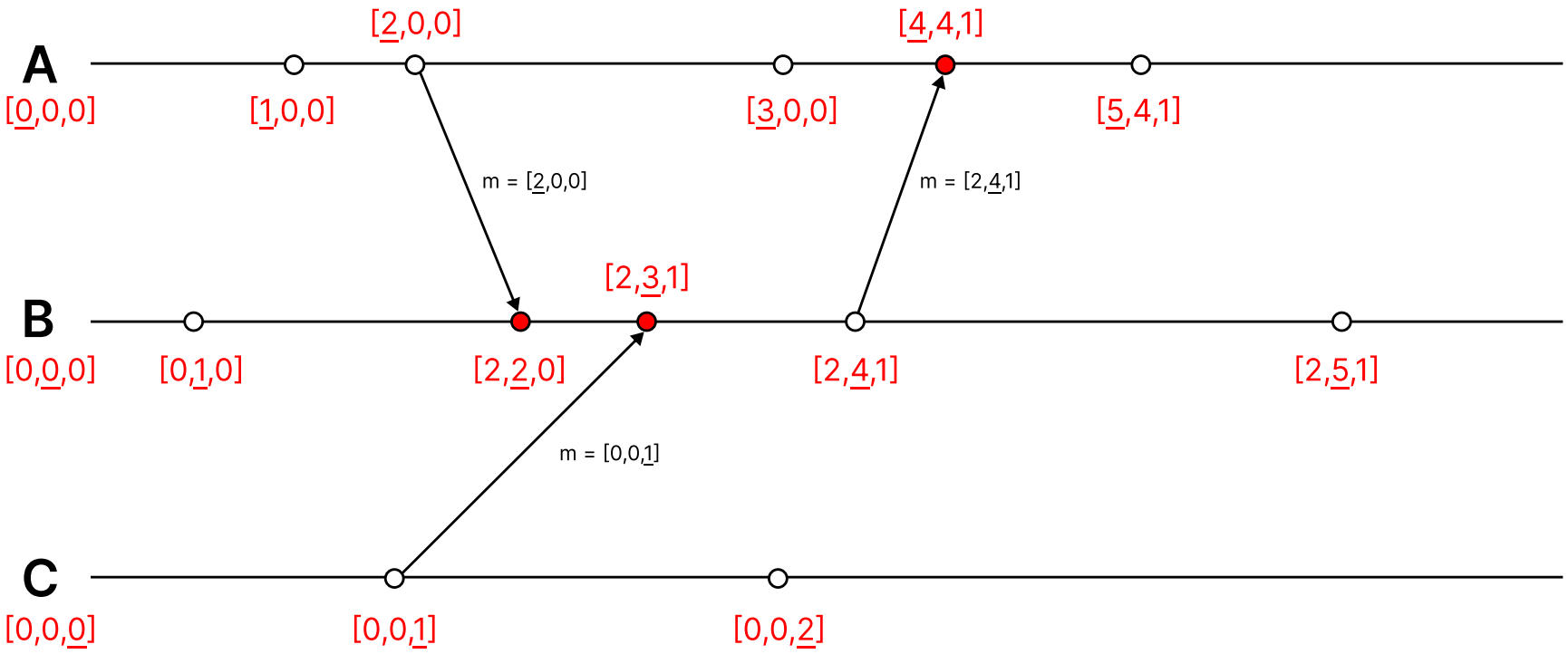
```
If:
    Vx = [3, 5, 2] // Vx[0] < Vy[0] (3 <
4)  Vy = [4, 3, 2] // Vx[1] > Vy[1] (5 >
3)
// Vx and Vy are concurrent
```

Logical clocks: Steps of Vector clock

1. All clock components are initialized to the smallest value (e.i: 0)
2. Each primitive event in a node increments its local clock. vcx[x] = vcx[x] + 1
3. A node timestamps its vector clock in every outgoing message.
4. Upon message receiving, the receiver node updates its local vector by taking **a component-wise maximum** with the new vector clock. And increasing one tick at its own time



Logical clocks: Vector clock example



- [Blank dots]: Events
- [Red filled dots]: Events in which vector clocks are synchronized after a message reception

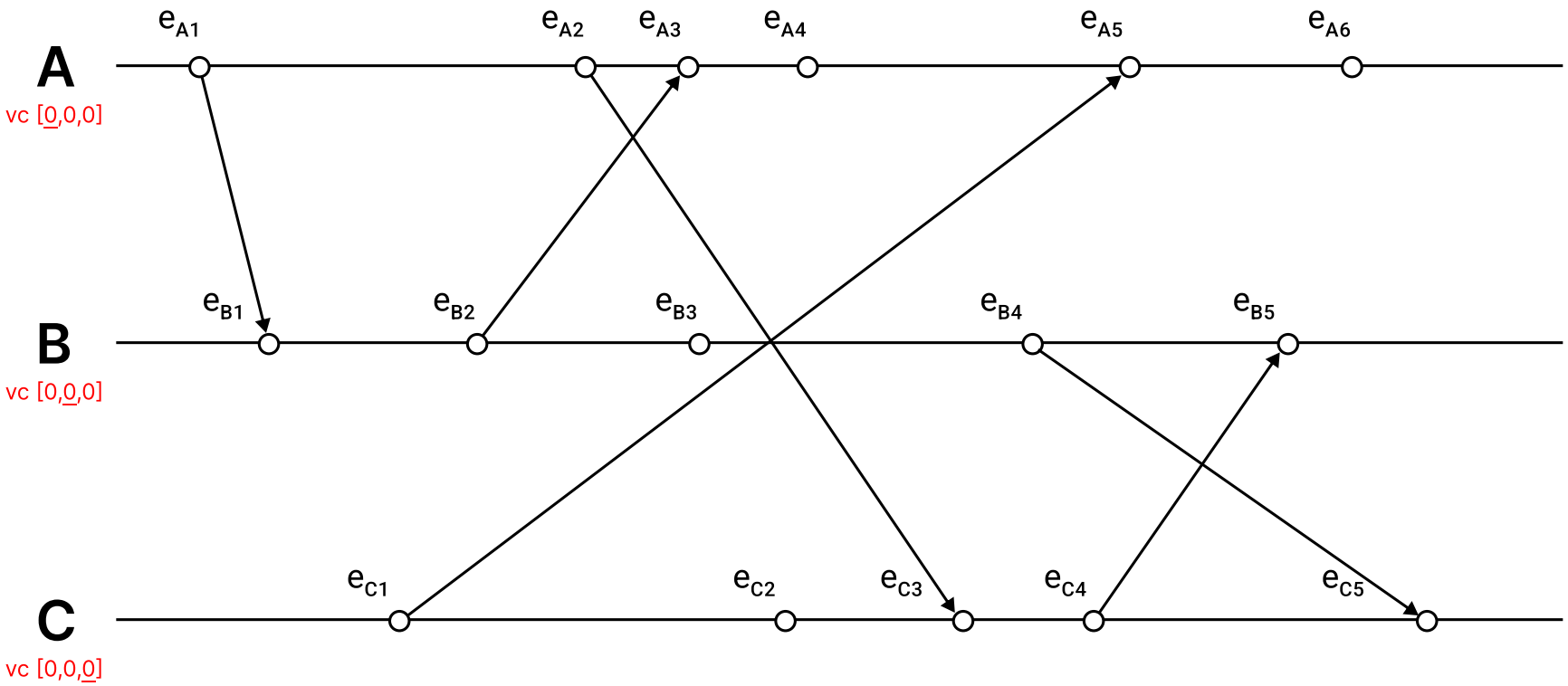
Logical clocks: Vector clock limitation

1. A vector clock consists of one entry per node in the system. Which can potentially lead to **large vectors** for large systems.

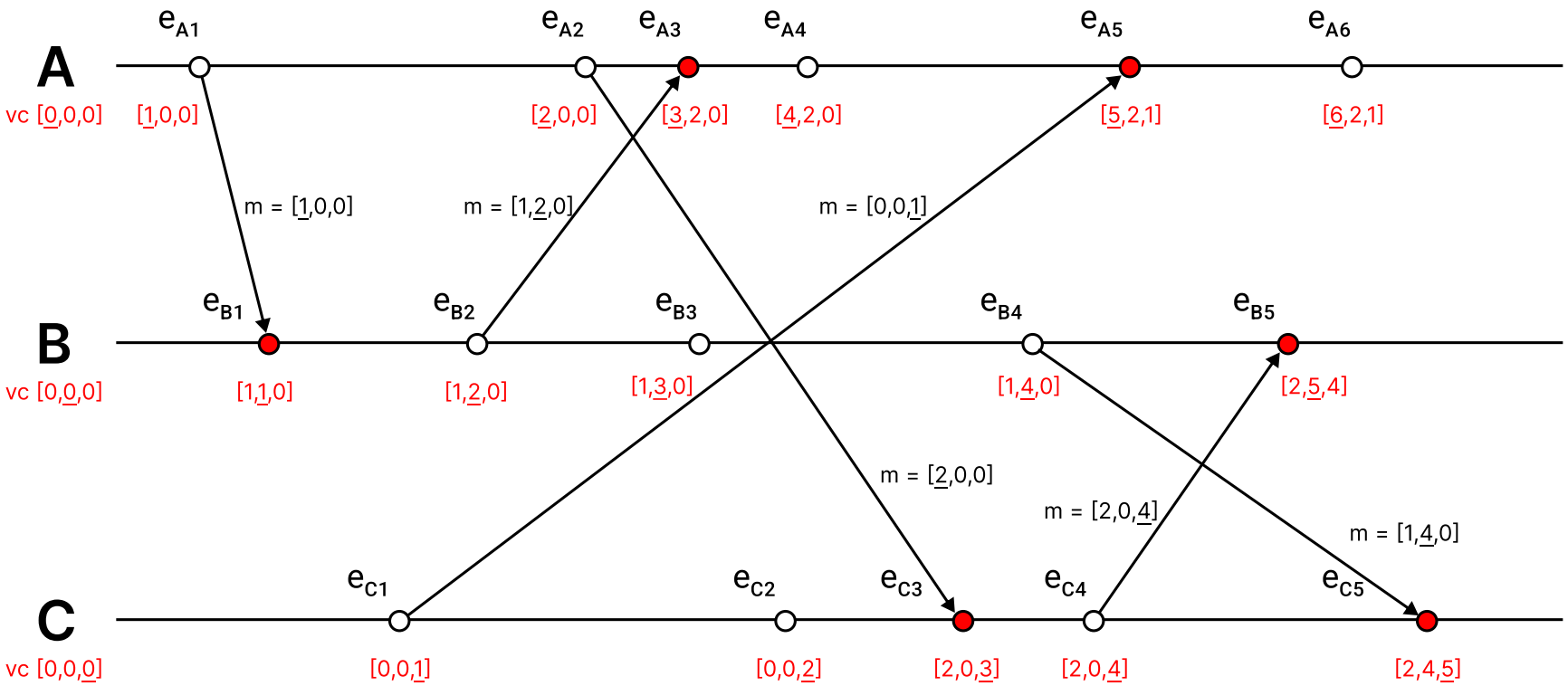
2. Vector clocks are attached to the message each time two nodes interact with each other, which increases **communication overhead**.

3. **Processing overhead**, all vector clock components need to be synchronized on every message reception.

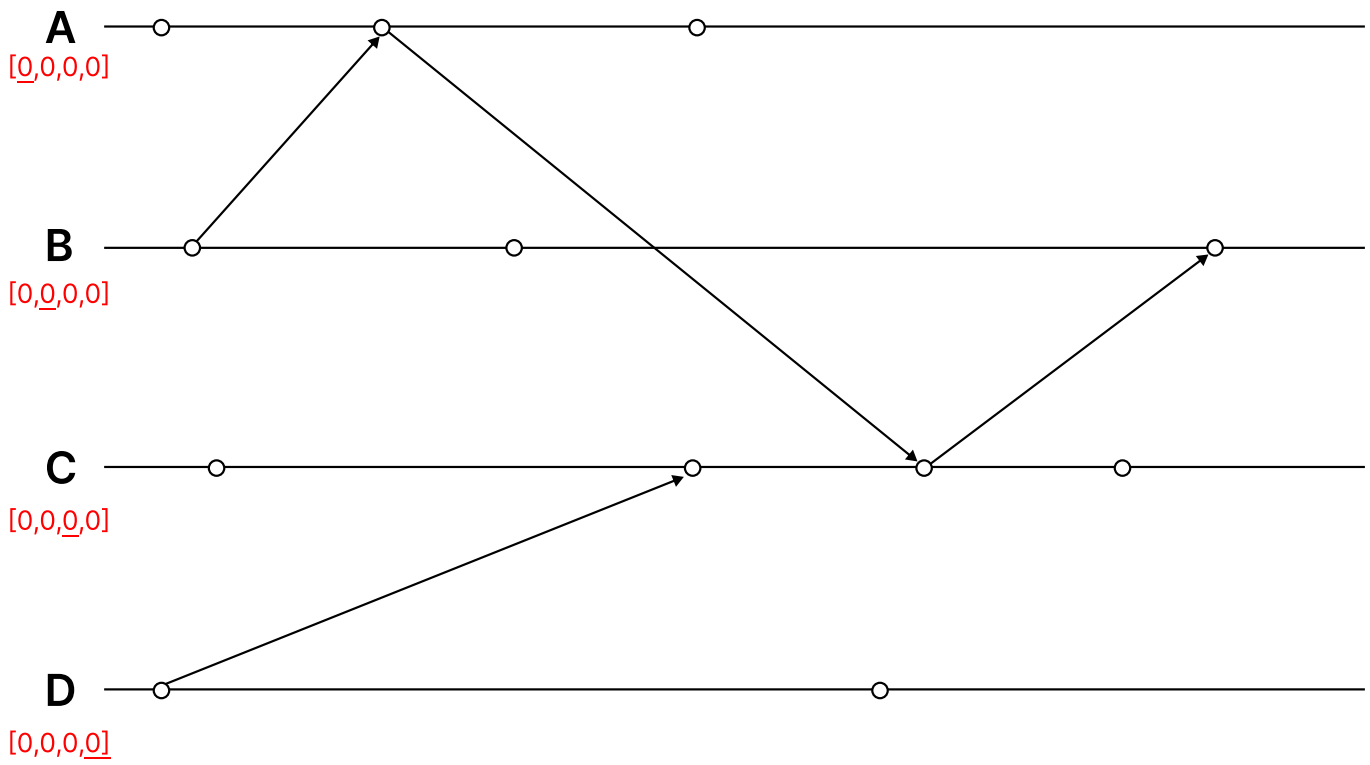
Logical clocks: Vector clock exercise 1



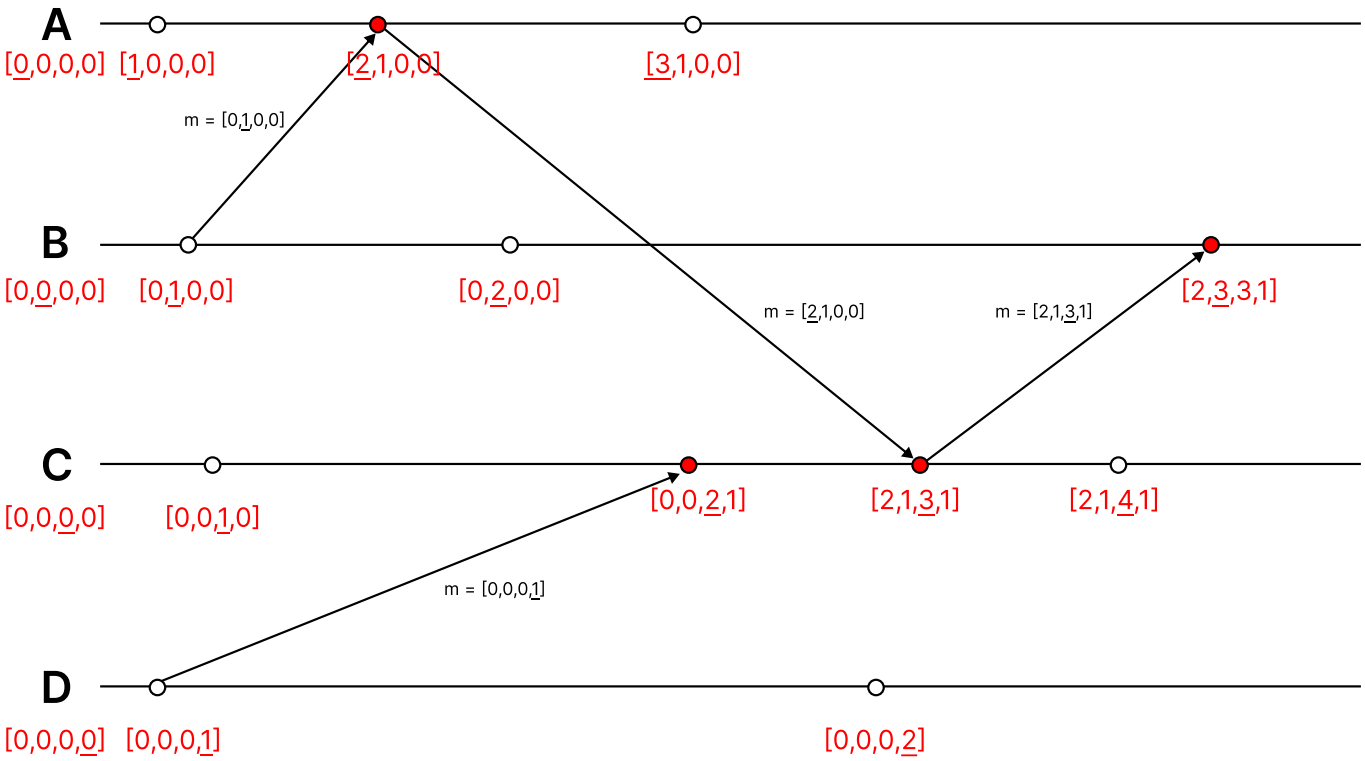
Logical clocks: Vector clock solution 1



Logical clocks: Vector clock exercise 2



Logical clocks: Vector clock solution 2



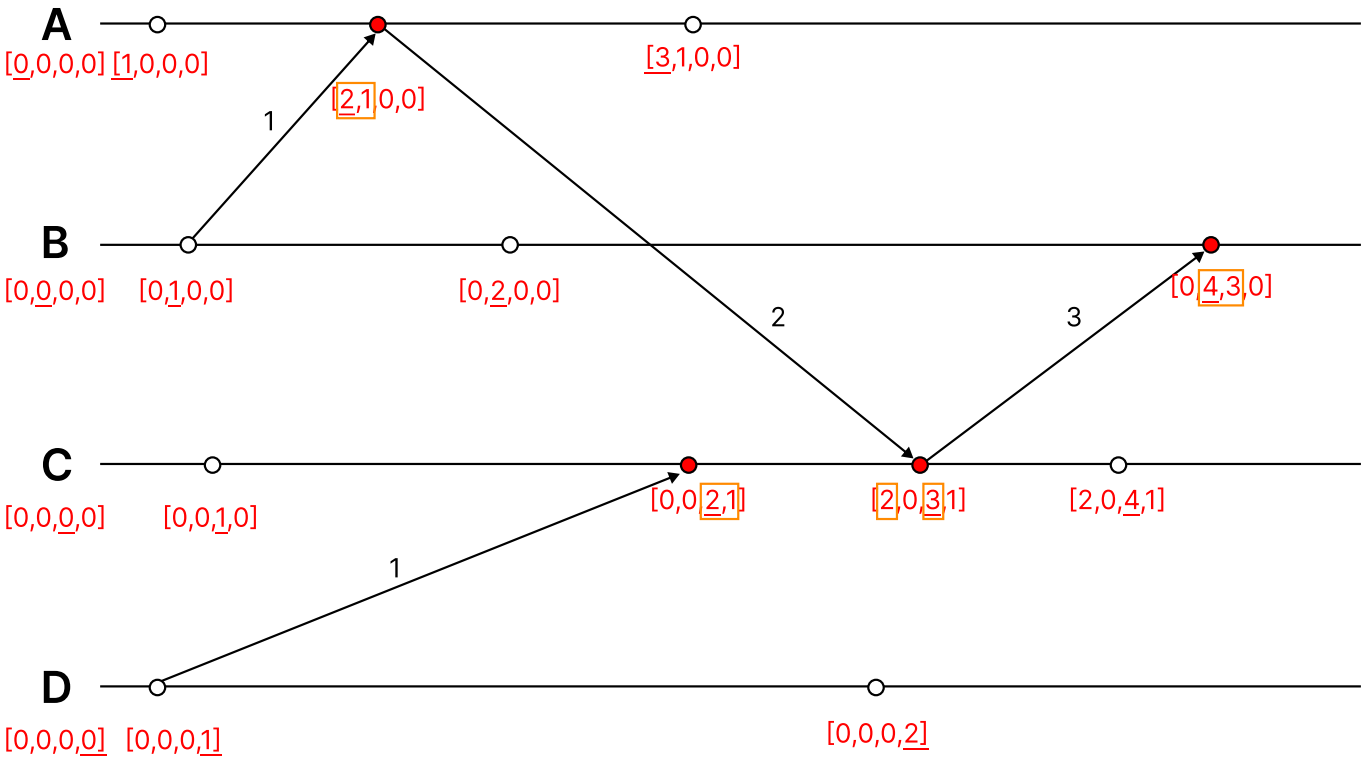
Logical clocks: Direct dependency clock

Vector clocks require O(N) integers to be sent with every message.

Direct clocks are a **weaker** version of vector clocks and requires a single integer per message.

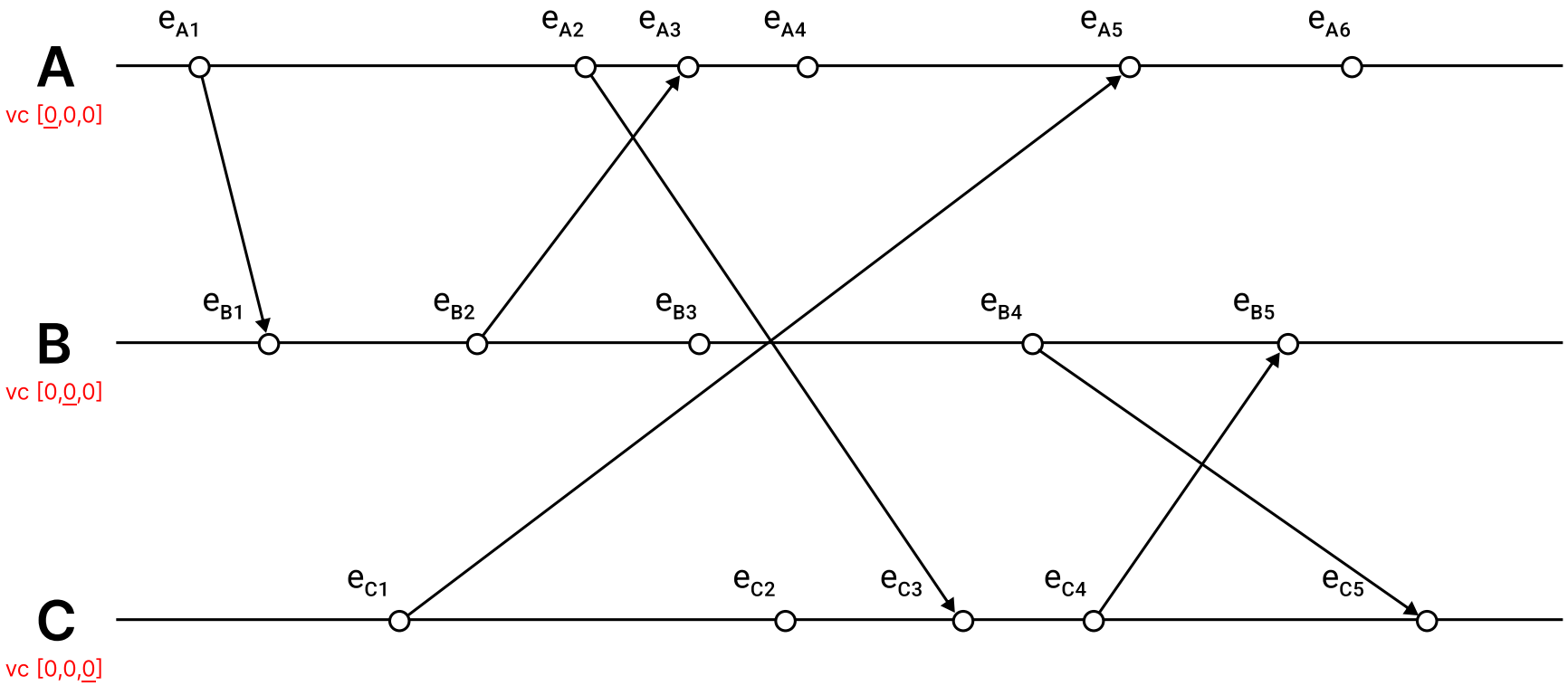
1. On a send event the process sends only its clock component.
2. On message reception, the receiver performs two updates:
    - Syncs own clock
    - Syncs the sender's clock.

Logical clocks: Direct dependency clock example

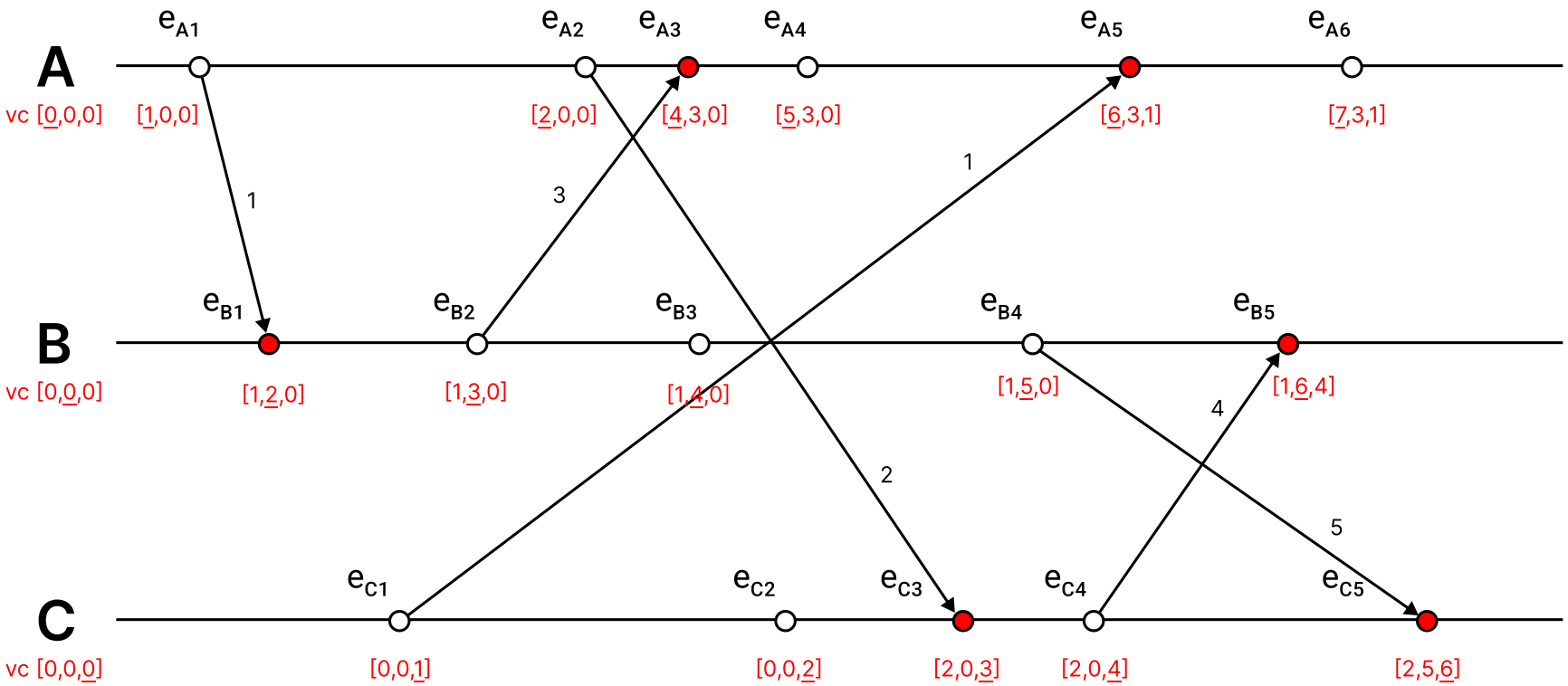


- These components are updated in every sync
- Blank dots are local events
- Red filled dots are when vector clocks are synchronized after a message passing reception

Logical clocks: Direct dependency clock exercise 1



Logical clocks: Direct dependency clock solution 1



Appendix: Logical clocks: Vector clock

```java
public class VectorClock {
    private int[] clock;
    int myId;
    int N;

    public VectorClock(int numProc, int id) {
        this.myId = id;
        this.N = numProc;
        this.clock = new int[numProc];
        for (int i = 0; i < numProc; i++) this.clock[i] = 0;
    }

    public int getValue(int i) {
        return this.clock[i];
    }

    public void tick() {
        this.clock[myId]++;
    }

    // Increase before sending & attach new vector clock to the message
    public void sendAction() {
        tick();
    }

    public void receiveAction(int[] sentValue) {
        for (int i = 0; i < this.N; i++)
            this.clock[i] = Util.max(this.clock[i], sentValue[i]);
        this.clock[myId]++;
    }
}
```

Appendix: Logical clocks: Direct dependency clock

```java
public class DirectClock {
    private int[] clock;
    int myId;

    public DirectClock(int numProc, int id) {
        this.myId = id;
        this.clock = new int[numProc];
        for (int i = 0; i < numProc; i++) this.clock[i] = 0;
    }

    public int getValue(int i) {
        return clock[i];
    }

    public void tick() {
        this.clock[myId]++;
    }

    public void sendAction() {
        tick();
    }

    public void receiveAction(int sender, int sentValue) {
        this.clock[sender] = Util.max(this.clock[sender], sentValue);
        this.clock[myId] = Util.max(this.clock[myId], sentValue) + 1;
    }
}
```

Recommended references

Network Time Protocol Rev3 RFC 1305
https://www.rfc-editor.org/rfc/rfc1305.pdf

Time, clocks, and the ordering of events in a Distributed System
https://lamport.azurewebsites.net/pubs/time-clocks.pdf